How to add an MCP server to Voice Live (preview)

> **Note**
>
> This feature is currently in public preview. This preview is provided without a service-level agreement, and is not recommended for production workloads. Certain features might not be supported or might have constrained capabilities. For more information, see Supplemental Terms of Use for Microsoft Azure Previews .

Introduction

Voice Live supports connecting to remote Model Context Protocol (MCP) servers during a voice session. MCP integration enables the model to discover and invoke tools hosted on external services, such as documentation search, wiki lookup, or custom APIs, and incorporate tool results into spoken responses.

MCP server integration differs from function calling in these ways:

Expand table

Aspect	Function calling	MCP server
Tool execution	Client-side	Server-side (managed by Voice Live)
Tool discovery	Client defines tools explicitly	Voice Live auto-discovers tools from MCP endpoint
Approval model	Not applicable	Configurable: `"always"` (default), `"never"`, or per-tool dictionary
API version required	`2025-10-01`	`2026-01-01-preview` or later

Key concepts

- **`MCPServer` definition**: Declare one or more MCP endpoints in the session configuration with `server_label`, `server_url`, and optional `allowed_tools`, `headers`, `authorization`, and `require_approval`.
- **Tool discovery**: On session start, Voice Live calls each MCP server's tool listing endpoint and emits `mcp_list_tools` events.
- **Tool invocation**: When the model decides to call an MCP tool, the service handles execution and streams `response.mcp_call` events.
- **Approval flow**: When `require_approval` is set to `"always"` (the default), the client receives an `mcp_approval_request` conversation item and must respond with an `mcp_approval_response` before the call executes. Set `require_approval` to `"never"` for automatic execution, or use a per-tool dictionary to mix modes on the same server.

Approval modes

The `require_approval` property on each `MCPServer` controls whether tool calls need client-side approval before execution. It accepts a string or a per-tool dictionary.

Expand table

Mode	Value	Behavior
Always (default)	`"always"`	Every tool call sends an `mcp_approval_request` to the client. The call doesn't execute until the client responds with `mcp_approval_response` and `approve=true`.
Never	`"never"`	Tool calls execute automatically. No approval event is sent.
Per-tool	`{"always": ["tool_a"], "never": ["tool_b", "tool_c"]}`	Each tool is assigned an approval mode individually. Tools not listed in either key default to `"always"`.

When to use each mode:

- **`"always"`** — Use for tools that perform write operations, access sensitive data, or incur costs. The voice samples auto-approve subsequent calls to the same server within the same turn to reduce repeated prompts.
- **`"never"`** — Use for read-only lookups, search APIs, or trusted internal tools where user confirmation adds latency without security benefit.
- **Per-tool dictionary** — Use when a single MCP server exposes a mix of read-only and

write tools. For example, a documentation server might allow `search_docs` without approval but require approval for `submit_feedback`.

> **Note**
>
> In voice scenarios, each approval triggers a conversational prompt. Configure `require_approval` carefully to balance security with conversation flow. See **Voice-native approval** for implementation patterns.

For the full MCP event and type reference, see Voice Live API reference (2026-01-01-preview).

Learn how to connect remote MCP servers to a Voice Live session using the VoiceLive SDK for Python. This article builds on the Quickstart: Create a Voice Live real-time voice agent with MCP server integration.

Reference documentation | Package (PyPi) | Additional samples on GitHub

Follow the how-to below or get the full sample code:

Voice Live MCP sample

Prerequisites

- An Azure subscription. Create one for free .
- Python 3.10 or later version . If you don't have a suitable version of Python installed, you can follow the instructions in the VS Code Python Tutorial for the easiest way of installing Python on your operating system.
- A Microsoft Foundry resource created in one of the supported regions. For more information about region availability, see the Voice Live overview documentation.
- `azure-ai-voicelive` package version 1.0.0b5 or later (MCP support requires `api_version="2026-01-01-preview"`).
- Assign the `Cognitive Services User` role to your user account. You can assign roles in the Azure portal under **Access control (IAM)** > **Add role assignment**.

> **Tip**

To use Voice Live with MCP, you don't need to deploy an audio model with your Foundry resource. Voice Live is fully managed, and the model is automatically deployed for you. For more information about model availability, see the **Voice Live overview documentation**.

Prepare the environment

Complete the Voice Live quickstart to set up your environment, configure authentication, and test your first Voice Live conversation.

MCP integration concepts

MCP server definition

Use the `MCPServer` class to declare each remote MCP endpoint. At minimum, provide `server_label` (a display name) and `server_url` (the MCP endpoint URL). Optionally restrict available tools with `allowed_tools` and configure the approval mode.

Approval modes

Control whether MCP tool calls require user approval before execution:

- `require_approval="never"`: The tool executes automatically when the model invokes it.
- `require_approval="always"` (default): The client receives an `mcp_approval_request` and must respond before the tool runs.
- Per-tool dictionary: Set `require_approval={"never": ["tool_a"], "always": ["tool_b"]}` for granular control.

API version requirement

MCP support requires `api_version="2026-01-01-preview"` or later. Pass this value in the `connect()` call.

Define MCP servers

Define the MCP servers that Voice Live can use during the session. Each server is an `MCPServer` instance added to the tools list in the session configuration.

The following code defines two MCP servers: one with automatic tool execution and one that requires user approval before running.

Python

```python
# Define MCP servers that Voice Live can use during the session.
# Each server is an MCPServer instance added to the tools list.
mcp_tools: list[Tool] = [
    MCPServer(
        server_label="deepwiki",
        server_url="https://mcp.deepwiki.com/mcp",
        allowed_tools=["read_wiki_structure", "ask_question"],
        require_approval="never",
    ),
    MCPServer(
        server_label="azure_doc",
        server_url="https://learn.microsoft.com/api/mcp",
        require_approval="always",
    ),
]
```

In this sample:

- The `deepwiki` server allows only `read_wiki_structure` and `ask_question` tools, with `require_approval="never"` for automatic execution.
- The `azure_doc` server allows all tools on the endpoint, with `require_approval="always"` so users can review each call before execution.

Configure the session with MCP tools

Pass the MCP server definitions to the `RequestSession` tools list alongside your voice, modality, and turn-detection settings.

Python

```python
async def _setup_session(self, mcp_tools: list[Tool]):
    """Configure the VoiceLive session with MCP tools."""
    logger.info("Setting up voice conversation session with MCP tools...")

    # Create voice configuration
    voice_config: Union[AzureStandardVoice, str]
```

```
    if "-" in self.voice or ":" in self.voice:
        voice_config = AzureStandardVoice(name=self.voice)
    else:
        voice_config = self.voice

    # Create turn detection configuration
    turn_detection_config = ServerVad(
        threshold=0.5,
        prefix_padding_ms=300,
        silence_duration_ms=500)

    # Create session configuration with MCP tools in the tools list
    session_config = RequestSession(
        modalities=[Modality.TEXT, Modality.AUDIO],
        instructions=self.instructions,
        voice=voice_config,
        input_audio_format=InputAudioFormat.PCM16,
        output_audio_format=OutputAudioFormat.PCM16,
        turn_detection=turn_detection_config,
        input_audio_echo_cancellation=AudioEchoCancellation(),
        input_audio_noise_reduction=AudioNoiseReduction(type="azure_deep-
_noise_suppression"),
        tools=mcp_tools,
        tool_choice=ToolChoiceLiteral.AUTO,
        input_audio_transcription=AudioInputTranscriptionOptions(
            model="azure-speech" if "realtime" not in self.model.lower()
else "whisper-1"
        ),
    )

    # Interim response bridges latency during MCP tool calls, but is only
    # supported on non-realtime model pipelines (e.g. gpt-4o-mini).
    if "realtime" not in self.model.lower():
        session_config.interim_response = LlmInterimResponseConfig(
            triggers=[InterimResponseTrigger.TOOL, InterimResponseTrig-
ger.LATENCY],
            latency_threshold_ms=100,
            instructions="Create friendly interim responses indicating wait
time due to "
                         "ongoing processing, if any. Do not include in all
responses! "
                         "Do not say you don't have real-time access to in-
formation when "
                         "calling tools!",
        )
        logger.info("Interim response enabled for model %s", self.model)
    else:
        logger.info("Interim response skipped — not supported on realtime
pipeline (%s)", self.model)

    conn = self.connection
    assert conn is not None
    await conn.session.update(session=session_config)
    logger.info("Session configuration with MCP tools sent")
```

In this sample:

- `RequestSession` bundles MCP tools with audio format, voice, and turn detection settings.
- `connection.session.update(session=session_config)` sends the full configuration to Voice Live.
- Voice Live automatically discovers available tools from each MCP server after the session starts.

Handle MCP events

Process MCP-specific events in the event loop. The key events are:

- `CONVERSATION_ITEM_CREATED` with `ItemType.MCP_CALL`: An MCP tool call was triggered by the model.
- `RESPONSE_MCP_CALL_COMPLETED`: The MCP call completed successfully.
- `RESPONSE_MCP_CALL_FAILED`: The MCP call failed.
- `CONVERSATION_ITEM_CREATED` with `ItemType.MCP_APPROVAL_REQUEST`: The server is requesting approval for a tool call.
- `CONVERSATION_ITEM_CREATED` with `ItemType.MCP_LIST_TOOLS`: Tool discovery completed for a server.

Python

```python
async def _handle_event(self, event):
    """Handle different types of events from VoiceLive, including MCP events."""
    ap = self.audio_processor
    conn = self.connection
    assert ap is not None
    assert conn is not None

    if event.type == ServerEventType.SESSION_UPDATED:
        logger.info("Session ready: %s", event.session.id)
        await write_conversation_log(f"SessionID: {event.session.id}")
        await write_conversation_log(f"Model: {event.session.model}")
        await write_conversation_log(f"Voice: {event.session.voice}")
        await write_conversation_log("")
        self.session_ready = True
        ap.start_capture()

    elif event.type == ServerEventType.INPUT_AUDIO_BUFFER_SPEECH_STARTED:
```

```
        logger.info("User started speaking - stopping playback")
        print("🎤 Listening...")
        ap.skip_pending_audio()
        # Approval call counter is NOT reset on speech - it tracks the
        # lifecycle of a task (reset on denial or after results are spoken)
        # But approved-servers-this-turn resets when user starts a new topic
        if self._pending_approval is None and self._mcp_call_in_progress <=
0:
            self._approved_servers_this_turn.clear()

        # Clear deferred response flags if no MCP calls are in progress.
        # Prevents stale _needs_response_create from re-triggering result
        # playback after the user interrupts.
        if self._mcp_call_in_progress <= 0:
            self._needs_response_create = False
            self._mcp_results_pending = False

        if self._active_response and not self._response_api_done:
            try:
                await conn.response.cancel()
            except Exception as e:
                if "no active response" not in str(e).lower():
                    logger.warning("Cancel failed: %s", e)

        # If an MCP call is running, mark current calls as stale (user is
moving on)
        # and let the user know it's still in progress
        if self._mcp_call_in_progress > 0 and self._pending_approval is
None:
            self._stale_mcp_items.update(self._active_mcp_items)
            logger.info("User spoke during MCP call - marking %d calls as
stale", len(self._active_mcp_items))
            try:
                await conn.conversation.item.create(
                    item=MessageItem(
                        role="system",
                        content=[InputTextContentPart(
                            text="A tool call is still running in the back-
ground. The user just spoke. "
                                 "Respond to what the user said. If a tool
result arrives later, "
                                 "briefly introduce it as a late result from
an earlier request."
                        )],
                    )
                )
            except Exception as e:
                logger.warning("Failed to inject MCP status update: %s", e)

    elif event.type == ServerEventType.INPUT_AUDIO_BUFFER_SPEECH_STOPPED:
        logger.info("User stopped speaking")
        print("🤔 Processing...")

    elif event.type == ServerEventType.RESPONSE_CREATED:
```

```
            logger.info("Assistant response created")
            self._active_response = True
            self._response_api_done = False

        elif event.type == ServerEventType.RESPONSE_AUDIO_DELTA:
            ap.queue_audio(event.delta)

        elif event.type == ServerEventType.RESPONSE_AUDIO_DONE:
            logger.info("Assistant finished speaking")
            print("🎤 Ready for next input...")

        elif event.type == ServerEventType.RESPONSE_TEXT_DONE:
            text = event.text if hasattr(event, 'text') else event.get("text",
"")
            print(f"🤖 Assistant text:\t{text}")
            await write_conversation_log(f"Assistant Text Response:\t{text}")

        elif event.type == ServerEventType.RESPONSE_AUDIO_TRANSCRIPT_DONE:
            transcript = event.transcript if hasattr(event, 'transcript') else
event.get("transcript", "")
            print(f"🤖 Assistant audio transcript:\t{transcript}")
            await write_conversation_log(f"Assistant Audio Response:\t{tran-
script}")

        elif event.type == ServerEventType.RESPONSE_DONE:
            logger.info("Response complete")
            await write_conversation_log("--- Response complete ---")
            self._active_response = False
            self._response_api_done = True

            # If an approval prompt needs to be injected, do it now that no re-
sponse is active
            if self._approval_prompt_needed and self._pending_approval is not
None:
                self._approval_prompt_needed = False
                await self._send_approval_voice_prompt(self._pending_approval,
conn)
            # If MCP results are pending and all calls are now done, create re-
sponse
            elif self._mcp_results_pending and self._mcp_call_in_progress <= 0
and self._pending_approval is None:
                self._mcp_results_pending = False
                try:
                    await conn.response.create()
                except Exception:
                    pass
            # If a response.create was deferred due to collision, retry now
            elif self._needs_response_create:
                self._needs_response_create = False
                try:
                    await conn.response.create()
                except Exception:
                    pass  # Best-effort retry
```

```
    elif event.type == ServerEventType.CONVERSATION_ITEM_INPUT_AUDIO_TRAN-
SCRIPTION_COMPLETED:
        transcript = event.transcript if hasattr(event, 'transcript') else
event.get("transcript", "")
        logger.info("User said: %s", transcript)
        print(f"👤 You said:\t{transcript}")
        await write_conversation_log(f"User Input:\t{transcript}")

        # Interpret as an approval answer if we have a pending approval —
        # whether or not the prompt has finished speaking. This allows the
        # user to barge in with "yes" without waiting for the full prompt.
        if self._pending_approval is not None:
            await self._resolve_voice_approval(transcript, conn)

    elif event.type == ServerEventType.ERROR:
        msg = event.error.message
        # Reset response state — errors can terminate a response without
RESPONSE_DONE
        self._active_response = False
        self._response_api_done = True
        if "Cancellation failed: no active response" not in msg:
            if "interim response" in msg.lower():
                logger.warning("Interim response not supported with this
model pipeline (non-fatal)")
            elif "active response" in msg.lower():
                logger.debug("Response collision (expected during MCP flow):
%s", msg)
            else:
                logger.error("VoiceLive error: %s", msg)
                print(f"Error: {msg}")
                await write_conversation_log(f"ERROR: {msg}")

    # MCP-specific events
    elif event.type == ServerEventType.MCP_LIST_TOOLS_IN_PROGRESS:
        logger.info("MCP list tools in progress for %s", event.item_id)

    elif event.type == ServerEventType.MCP_LIST_TOOLS_COMPLETED:
        logger.info("MCP list tools completed for %s", event.item_id)
        print("🔧 MCP tools discovered successfully")
        await write_conversation_log("MCP tools discovered successfully")

    elif event.type == ServerEventType.MCP_LIST_TOOLS_FAILED:
        logger.error("MCP list tools failed for %s", event.item_id)
        print("❌ MCP tool discovery failed")
        await write_conversation_log("ERROR: MCP tool discovery failed")

    elif event.type == ServerEventType.RESPONSE_MCP_CALL_IN_PROGRESS:
        logger.info("MCP call in progress for %s", event.item_id)
        print("⏳ MCP tool call in progress...")
        await write_conversation_log(f"MCP call in progress: {even-
t.item_id}")
        self._mcp_call_in_progress += 1
        self._active_mcp_items.add(event.item_id)
        self._start_mcp_stall_timer(conn)
```

```
    elif event.type == ServerEventType.RESPONSE_MCP_CALL_COMPLETED:
        item_id = event.item_id
        self._mcp_call_in_progress = max(0, self._mcp_call_in_progress - 1)
        self._active_mcp_items.discard(item_id)
        self._cancel_mcp_stall_timer()
        if item_id in self._handled_mcp_completions:
            logger.debug("Ignoring duplicate MCP completion for %s",
item_id)
        else:
            self._handled_mcp_completions.add(item_id)
            is_stale = item_id in self._stale_mcp_items
            self._stale_mcp_items.discard(item_id)
            logger.info("MCP call completed for %s (stale=%s)", item_id,
is_stale)
            await write_conversation_log(f"MCP call completed: {item_id}
(stale={is_stale})")
            await self._handle_mcp_call_completed(event, conn,
is_stale=is_stale)

    elif event.type == ServerEventType.RESPONSE_MCP_CALL_FAILED:
        item_id = event.item_id
        logger.error("MCP call failed for %s", item_id)
        print("❌ MCP tool call failed")
        await write_conversation_log(f"ERROR: MCP call failed: {item_id}")
        self._mcp_call_in_progress = max(0, self._mcp_call_in_progress - 1)
        self._active_mcp_items.discard(item_id)
        self._stale_mcp_items.discard(item_id)
        self._cancel_mcp_stall_timer()
        # Kick the model to inform the user the tool call failed
        try:
            await conn.response.create()
        except Exception as e:
            if "active response" not in str(e).lower():
                logger.warning("Failed to create response after MCP failure:
%s", e)

    elif event.type == ServerEventType.CONVERSATION_ITEM_CREATED:
        logger.info("Conversation item created: id=%s, type=%s", even-
t.item.id, event.item.type)
        if event.item.type == ItemType.MCP_LIST_TOOLS:
            logger.info("MCP list tools item: server_label=%s", even-
t.item.server_label)
        elif event.item.type == ItemType.MCP_CALL:
            await self._handle_mcp_call_arguments(event, conn)
        elif event.item.type == ItemType.MCP_APPROVAL_REQUEST:
            await self._handle_mcp_approval_request(event, conn)
    else:
        logger.debug("Unhandled event type: %s", event.type)
```

In this sample:

- `_handle_mcp_call_arguments` waits for the full arguments to stream in via `RESPONSE_MCP_CALL_ARGUMENTS_DONE`, then waits for the response to complete.
- `_handle_mcp_call_completed` receives the tool output and triggers a new response so the model can incorporate the result into its next spoken reply.

Handle approval requests

When a server is configured with `require_approval="always"`, client code must handle the approval flow. Instead of blocking on console input, inject a system message so the model asks the user verbally and parse the spoken response.

Python

```
async def _handle_mcp_approval_request(self, conversation_created_event,
connection):
    """Handle MCP approval request by asking the user via voice."""
    if not isinstance(conversation_created_event, ServerEventConversation-
ItemCreated):
        logger.error("Expected ServerEventConversationItemCreated")
        return
    if not isinstance(conversation_created_event.item, ResponseMCPApproval-
RequestItem):
        logger.error("Expected ResponseMCPApprovalRequestItem")
        return

    mcp_approval_item = conversation_created_event.item
    approval_id = mcp_approval_item.id
    server_label = mcp_approval_item.server_label
    function_name = mcp_approval_item.name

    if not approval_id:
        logger.error("MCP approval item missing ID")
        return

    # Auto-deny after too many calls to the same server in one task.
    # This prevents infinite tool-call loops in voice UX.
    MAX_APPROVAL_CALLS_PER_TASK = 3
    current_count = self._approval_call_count.get(server_label, 0)
    if current_count >= MAX_APPROVAL_CALLS_PER_TASK:
        logger.info("Auto-denying %s — reached %d calls this task", func-
tion_name, current_count)
        print(f"   Auto-denied: {server_label}/{function_name} (max {MAX_AP-
PROVAL_CALLS_PER_TASK} calls reached)")
        try:
            await connection.conversation.item.create(
                item=MCPApprovalResponseRequestItem(approval_request_id=ap-
proval_id, approve=False)
            )
```

```
            except Exception as e:
                logger.warning("Failed to send auto-deny: %s", e)
            return

        # Auto-approve if user already approved this server earlier in the same
turn.
        # This avoids repeated approval prompts for consecutive calls to the
same service.
        if server_label in self._approved_servers_this_turn:
            logger.info("Auto-approving %s - server already approved this turn",
function_name)
            print(f"   Auto-approved: {server_label}/{function_name} (already
approved this turn)")
            try:
                await connection.conversation.item.create(
                    item=MCPApprovalResponseRequestItem(approval_request_id=ap-
proval_id, approve=True)
                )
            except Exception as e:
                logger.warning("Failed to send auto-approve: %s", e)
            return

        # If another approval is already pending, queue this one
        if self._pending_approval is not None:
            logger.info("Queuing approval for %s - another is already pending",
function_name)
            self._approval_queue.append({
                "approval_id": approval_id,
                "server_label": server_label,
                "function_name": function_name,
            })
            return

        logger.info("MCP approval request: server=%s tool=%s", server_label,
function_name)
        print(f"\n🔐 MCP Approval Request (voice-based):")
        print(f"   Server: {server_label}  Tool: {function_name}")

        # Store the pending approval. If no response is currently active,
        # send the voice prompt immediately. Otherwise, defer it to
        # RESPONSE_DONE to avoid colliding with an active response.
        self._pending_approval = {
            "approval_id": approval_id,
            "server_label": server_label,
            "function_name": function_name,
        }

        if not self._active_response:
            await self._send_approval_voice_prompt(self._pending_approval, con-
nection)
        else:
            self._approval_prompt_needed = True

async def _send_approval_voice_prompt(self, pending: dict, connection):
```

```
    """Inject a system message asking the model to verbally request permis-
sion."""
    server = pending["server_label"]
    call_count = self._approval_call_count.get(server, 0)
    self._approval_call_count[server] = call_count + 1

    if call_count == 0:
        prompt = (
            "You MUST ask the user for explicit permission before proceed-
ing. "
            f'Say exactly: "I\'d like to search the {server} service for
information. '
            f'Do you approve? Please say yes or no."'
        )
    else:
        prompt = (
            "You MUST ask the user for permission again. "
            'Say exactly: "I need to do one more search to get complete in-
formation. '
            'Should I continue? Please say yes or no."'
        )

    try:
        await connection.conversation.item.create(
            item=MessageItem(
                role="system",
                content=[InputTextContentPart(text=prompt)],
            )
        )
        await connection.response.create()
    except Exception as e:
        logger.warning("Failed to send approval voice prompt: %s", e)

async def _resolve_voice_approval(self, transcript: str, connection):
    """Interpret the user's spoken response as approval or denial."""
    pending = self._pending_approval
    if pending is None:
        return

    text = transcript.strip().lower()

    # Match "yes" or "no" as whole words (word boundaries prevent false
    # positives from words like "yesterday" or "nobody").
    # Also accept "stop" and "cancel" as denial.
    approved = bool(re.search(r'\byes\b', text))
    denied = bool(re.search(r'\b(no|stop|cancel)\b', text))

    if not approved and not denied:
        # Ambiguous — ask again via the deferred prompt mechanism
        logger.info("Ambiguous approval response: %s", transcript)
        self._approval_prompt_needed = True
        return

    if approved and denied:
```

```
            # Conflicting signals — treat as denial for safety
            approved = False

        # Clear the pending state before sending the response
        self._pending_approval = None
        if approved:
            self._approved_servers_this_turn.add(pending["server_label"])
        else:
            self._approval_call_count.clear()  # Topic is over
            self._approved_servers_this_turn.discard(pending["server_label"])

        approval_response_item = MCPApprovalResponseRequestItem(
            approval_request_id=pending["approval_id"], approve=approved
        )
        try:
            await connection.conversation.item.create(item=approval_respon-
se_item)
        except Exception as e:
            logger.error("Failed to send approval response: %s", e)
            return
        logger.info("Voice approval resolved: %s for %s", approved, pend-
ing["function_name"])
        print(f"   Voice approval: {'Approved ✅' if approved else 'Denied
❌'}")
        await write_conversation_log(f"Voice approval: {'Approved' if approved
else 'Denied'} for {pending['server_label']}")

        # Process next queued approval, if any
        await self._process_next_approval(connection)

    async def _process_next_approval(self, connection):
        """Pop the next queued approval and ask via voice."""
        if not self._approval_queue:
            return
        next_approval = self._approval_queue.pop(0)
        self._pending_approval = next_approval

        # Send immediately if no response is active, otherwise defer
        if not self._active_response:
            await self._send_approval_voice_prompt(next_approval, connection)
        else:
            self._approval_prompt_needed = True
```

In this sample:

- The `mcp_approval_request` event contains `server_label`, `name` (tool name), and `arguments`.
- A system message instructs the model to verbally ask for permission.
- `MCPApprovalResponseRequestItem` sends the decision back to Voice Live with `approve=True` or `approve=False`.

Resolve voice-based approval

Parse the user's spoken transcript to determine approval. Use word-boundary regex to avoid false positives from words like "yesterday" or "nobody".

Python

```
elif event.type == ServerEventType.CONVERSATION_ITEM_INPUT_AUDIO_TRANSCRIP-
TION_COMPLETED:
    transcript = event.transcript if hasattr(event, 'transcript') else even-
t.get("transcript", "")
    logger.info("User said: %s", transcript)
    print(f"👤 You said:\t{transcript}")
    await write_conversation_log(f"User Input:\t{transcript}")

    # Interpret as an approval answer if we have a pending approval —
    # whether or not the prompt has finished speaking. This allows the
    # user to barge in with "yes" without waiting for the full prompt.
    if self._pending_approval is not None:
        await self._resolve_voice_approval(transcript, conn)
```

In this sample:

- The transcript from `CONVERSATION_ITEM_INPUT_AUDIO_TRANSCRIPTION_COMPLETED` is matched against `\byes\b` and `\b(no|stop|cancel)\b` patterns.
- Subsequent calls to the same server within the same turn are auto-approved to avoid repeated prompts.
- After a configurable maximum (for example, 3 approvals), further calls are auto-denied and the model responds with what it has.

Detect stalls during MCP tool calls

MCP tool calls can take several seconds. Use a repeating timer to proactively inform the user that the assistant is still waiting for results.

Python

```
MCP_STALL_MAX_NOTIFICATIONS = 3

def _start_mcp_stall_timer(self, connection):
    """Start a repeating timer that verbally updates the user if an MCP call
takes too long."""
    self._cancel_mcp_stall_timer()
```

```
    async def _stall_loop():
        stall_count = 0
        while self._mcp_call_in_progress > 0 and stall_count < self.MC-
P_STALL_MAX_NOTIFICATIONS:
            await asyncio.sleep(10)
            if self._mcp_call_in_progress <= 0:
                break
            stall_count += 1
            # Note: MCP calls cannot be cancelled via the API — only honest
            # status updates are possible until the server responds or times
out.
            msg = ("The tool call is still running. "
                   "Briefly reassure the user that you're still waiting for
results. "
                   "One short sentence only.")
            logger.info("MCP stall notification #%d", stall_count)
            try:
                await connection.conversation.item.create(
                    item=MessageItem(
                        role="system",
                        content=[InputTextContentPart(text=msg)],
                    )
                )
                await connection.response.create()
            except Exception as e:
                if "active response" in str(e).lower():
                    self._needs_response_create = True
                else:
                    logger.debug("Stall notification failed: %s", e)

    self._mcp_stall_task = asyncio.create_task(_stall_loop())

def _cancel_mcp_stall_timer(self):
    """Cancel the MCP stall timer if running."""
    if self._mcp_stall_task and not self._mcp_stall_task.done():
        self._mcp_stall_task.cancel()
    self._mcp_stall_task = None
```

In this sample:

- A 10-second interval timer injects system messages like "Tell the user you're still waiting" up to 3 times.
- The timer is cancelled when the MCP call completes or the user interrupts with barge-in.

Run the sample

1. Create the `mcp-quickstart.py` file with the following code:

Python

```python
# ------------------------------------------------------------------------
----
# Copyright (c) Microsoft Corporation. All rights reserved.
# Licensed under the MIT License.
# ------------------------------------------------------------------------
----

"""
FILE: mcp-quickstart.py

DESCRIPTION:
    This sample demonstrates how to use the Azure AI Voice Live SDK
with MCP
    (Model Context Protocol) server integration. It shows how to define
MCP
    servers, handle MCP tool call events, and implement an approval
flow for
    tool calls that require user consent.

USAGE:
    python mcp-quickstart.py --use-token-credential

    Set the environment variables with your own values before running
the sample:
    1) AZURE_VOICELIVE_ENDPOINT - The Azure VoiceLive endpoint
    2) AZURE_VOICELIVE_API_KEY  - The Azure VoiceLive API key (if not
using token credential)

REQUIREMENTS:
    - azure-ai-voicelive
    - python-dotenv
    - pyaudio (for audio capture and playback)
    - azure-identity (for token credential authentication)
"""

from __future__ import annotations
import os
import sys
import argparse
import asyncio
import base64
from datetime import datetime
import logging
import queue
import re
import signal
from typing import Union, Optional, TYPE_CHECKING, cast

from azure.core.credentials import AzureKeyCredential
from azure.core.credentials_async import AsyncTokenCredential
from azure.identity.aio import AzureCliCredential
```

```
from azure.ai.voicelive.aio import connect
from azure.ai.voicelive.models import (
    AudioEchoCancellation,
    AudioInputTranscriptionOptions,
    AudioNoiseReduction,
    AzureStandardVoice,
    InputAudioFormat,
    InputTextContentPart,
    InterimResponseTrigger,
    ItemType,
    LlmInterimResponseConfig,
    MCPApprovalResponseRequestItem,
    MCPServer,
    MessageItem,
    Modality,
    OutputAudioFormat,
    RequestSession,
    ResponseMCPApprovalRequestItem,
    ResponseMCPCallItem,
    ServerEventConversationItemCreated,
    ServerEventResponseMcpCallCompleted,
    ServerEventType,
    ServerVad,
    Tool,
    ToolChoiceLiteral,
)
from dotenv import load_dotenv
import pyaudio

if TYPE_CHECKING:
    from azure.ai.voicelive.aio import VoiceLiveConnection

# Change to the directory where this script is located
os.chdir(os.path.dirname(os.path.abspath(__file__)))

# Environment variable loading
load_dotenv('../.env', override=True)

# Set up logging
if not os.path.exists('logs'):
    os.makedirs('logs')

timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")

# Conversation log filename (separate from debug log)
_script_dir = os.path.dirname(os.path.abspath(__file__))
conversation_logfilename = f"conversation_{timestamp}.log"

logging.basicConfig(
    filename=f'logs/{timestamp}_voicelive.log',
    filemode="w",
    format='%(asctime)s:%(name)s:%(levelname)s:%(message)s',
    level=logging.INFO
```

```
)
logger = logging.getLogger(__name__)


class AudioProcessor:
    """
    Handles real-time audio capture and playback for the voice as-
sistant.

    Threading Architecture:
    - Main thread: Event loop and UI
    - Capture thread: PyAudio input stream reading
    - Send thread: Async audio data transmission to VoiceLive
    - Playback thread: PyAudio output stream writing
    """

    loop: asyncio.AbstractEventLoop

    class AudioPlaybackPacket:
        """Represents a packet that can be sent to the audio playback
queue."""
        def __init__(self, seq_num: int, data: Optional[bytes]):
            self.seq_num = seq_num
            self.data = data

    def __init__(self, connection):
        self.connection = connection
        self.audio = pyaudio.PyAudio()

        # Audio configuration - PCM16, 24kHz, mono
        self.format = pyaudio.paInt16
        self.channels = 1
        self.rate = 24000
        self.chunk_size = 1200  # 50ms

        # Capture and playback state
        self.input_stream = None

        self.playback_queue: queue.Queue[AudioProcessor.AudioPlayback-
Packet] = queue.Queue()
        self.playback_base = 0
        self.next_seq_num = 0
        self.output_stream: Optional[pyaudio.Stream] = None

        logger.info("AudioProcessor initialized with 24kHz PCM16 mono
audio")

    def start_capture(self):
        """Start capturing audio from microphone."""
        def _capture_callback(in_data, _frame_count, _time_info, _sta-
tus_flags):
            audio_base64 = base64.b64encode(in_data).decode("utf-8")
            asyncio.run_coroutine_threadsafe(
                self.connection.input_audio_buffer.append(audio=au-
```

```
dio_base64), self.loop
            )
            return (None, pyaudio.paContinue)

        if self.input_stream:
            return

        self.loop = asyncio.get_event_loop()

        try:
            self.input_stream = self.audio.open(
                format=self.format,
                channels=self.channels,
                rate=self.rate,
                input=True,
                frames_per_buffer=self.chunk_size,
                stream_callback=_capture_callback,
            )
            logger.info("Started audio capture")
        except Exception:
            logger.exception("Failed to start audio capture")
            raise

    def start_playback(self):
        """Initialize audio playback system."""
        if self.output_stream:
            return

        remaining = bytes()

        def _playback_callback(_in_data, frame_count, _time_info, _sta-
tus_flags):
            nonlocal remaining
            frame_count *= pyaudio.get_sample_size(pyaudio.paInt16)

            out = remaining[:frame_count]
            remaining_local = remaining[frame_count:]

            while len(out) < frame_count:
                try:
                    packet = self.playback_queue.get_nowait()
                except queue.Empty:
                    out = out + bytes(frame_count - len(out))
                    continue

                if not packet or not packet.data:
                    break

                if packet.seq_num < self.playback_base:
                    continue

                num_to_take = frame_count - len(out)
                out = out + packet.data[:num_to_take]
                remaining_local = packet.data[num_to_take:]
```

```
            remaining = remaining_local

            if len(out) >= frame_count:
                return (out, pyaudio.paContinue)
            else:
                return (out, pyaudio.paComplete)

        try:
            self.output_stream = self.audio.open(
                format=self.format,
                channels=self.channels,
                rate=self.rate,
                output=True,
                frames_per_buffer=self.chunk_size,
                stream_callback=_playback_callback
            )
            logger.info("Audio playback system ready")
        except Exception:
            logger.exception("Failed to initialize audio playback")
            raise

    def _get_and_increase_seq_num(self):
        seq = self.next_seq_num
        self.next_seq_num += 1
        return seq

    def queue_audio(self, audio_data: Optional[bytes]) -> None:
        """Queue audio data for playback."""
        self.playback_queue.put(
            AudioProcessor.AudioPlaybackPacket(
                seq_num=self._get_and_increase_seq_num(),
                data=audio_data))

    def skip_pending_audio(self):
        """Skip current audio in playback queue."""
        self.playback_base = self._get_and_increase_seq_num()

    def shutdown(self):
        """Clean up audio resources."""
        if self.input_stream:
            self.input_stream.stop_stream()
            self.input_stream.close()
            self.input_stream = None
        logger.info("Stopped audio capture")

        if self.output_stream:
            self.skip_pending_audio()
            self.queue_audio(None)
            self.output_stream.stop_stream()
            self.output_stream.close()
            self.output_stream = None
        logger.info("Stopped audio playback")
```

```
        if self.audio:
            self.audio.terminate()
        logger.info("Audio processor cleaned up")


class MCPVoiceAssistant:
    """Voice assistant with MCP server integration."""

    def __init__(
        self,
        endpoint: str,
        credential: Union[AzureKeyCredential, AsyncTokenCredential],
        model: str,
        voice: str,
        instructions: str,
    ):
        self.endpoint = endpoint
        self.credential = credential
        self.model = model
        self.voice = voice
        self.instructions = instructions
        self.connection: Optional["VoiceLiveConnection"] = None
        self.audio_processor: Optional[AudioProcessor] = None
        self.session_ready = False
        self._active_response = False
        self._response_api_done = False
        self._pending_approval: Optional[dict] = None  # Currently ac-
tive approval request
        self._approval_queue: list[dict] = []  # Queued approvals wait-
ing to be asked
        self._approval_prompt_needed = False  # True when we need to
inject the prompt at next RESPONSE_DONE
        self._mcp_call_in_progress = 0  # Count of active MCP tool
calls
        self._handled_mcp_completions: set = set()  # Deduplicate MCP
completion events
        self._needs_response_create = False  # Retry response.create at
next RESPONSE_DONE
        self._approval_call_count: dict[str, int] = {}  # Per-server
call count this turn
        self._mcp_item_to_server: dict = {}  # Map MCP item IDs to
server_label/function_name
        self._approval_servers: set = set()  # Server labels that re-
quire approval
        self._mcp_stall_task: Optional[asyncio.Task] = None  # Timer
for MCP stall detection
        self._active_mcp_items: set = set()  # Item IDs of currently
in-progress MCP calls
        self._stale_mcp_items: set = set()  # MCP calls the user has
moved on from
        self._approved_servers_this_turn: set = set()  # Servers user
already approved this turn
        self._mcp_results_pending = False  # True when MCP calls com-
pleted but response.create deferred
```

```
    async def start(self):
        """Start the voice assistant session with MCP support."""
        try:
            logger.info("Connecting to VoiceLive API with model %s",
self.model)

            # <define_mcp_servers>
            # Define MCP servers that Voice Live can use during the
session.
            # Each server is an MCPServer instance added to the tools
list.
            mcp_tools: list[Tool] = [
                MCPServer(
                    server_label="deepwiki",
                    server_url="https://mcp.deepwiki.com/mcp",
                    allowed_tools=["read_wiki_structure", "ask_ques-
tion"],
                    require_approval="never",
                ),
                MCPServer(
                    server_label="azure_doc",
                    server_url="https://learn.microsoft.com/api/mcp",
                    require_approval="always",
                ),
            ]
            # </define_mcp_servers>

            # Track which servers require approval for per-turn loop
prevention.
            # Servers with require_approval="always" are guarded to
avoid
            # repeated approval prompts in voice UX — a design decision
to keep
            # the voice conversation flow smooth. Servers with "never"
are allowed
            # to make multiple calls (e.g. DeepWiki's read_wiki_struc-
ture →
            # ask_question pattern) since they don't interrupt the
user.
            self._approval_servers = {
                s.server_label for s in mcp_tools
                if isinstance(s, MCPServer) and s.require_approval ==
"always"
            }

            # Connect with api_version="2026-01-01-preview" for MCP
support
            async with connect(
                endpoint=self.endpoint,
                credential=self.credential,
                model=self.model,
                api_version="2026-01-01-preview",
            ) as connection:
```

```
                self.connection = connection

                # Initialize audio processor
                ap = AudioProcessor(connection)
                self.audio_processor = ap

                # Configure session with MCP tools
                await self._setup_session(mcp_tools)

                # Start audio systems
                ap.start_playback()

                logger.info("Voice assistant with MCP ready! Start
speaking...")
                print("\n" + "=" * 70)
                print("🎤 VOICE ASSISTANT WITH MCP READY")
                print("Try saying:")
                print("  • 'What is the GitHub repo fastapi about?'")
                print("  • 'Search the Azure documentation for Voice
Live API.'")
                print("You may need to approve some MCP tool calls in
the console.")
                print("Press Ctrl+C to exit")
                print("=" * 70 + "\n")

                # Process events
                await self._process_events()
        finally:
            if self.audio_processor:
                self.audio_processor.shutdown()

    # <configure_session>
    async def _setup_session(self, mcp_tools: list[Tool]):
        """Configure the VoiceLive session with MCP tools."""
        logger.info("Setting up voice conversation session with MCP
tools...")

        # Create voice configuration
        voice_config: Union[AzureStandardVoice, str]
        if "-" in self.voice or ":" in self.voice:
            voice_config = AzureStandardVoice(name=self.voice)
        else:
            voice_config = self.voice

        # Create turn detection configuration
        turn_detection_config = ServerVad(
            threshold=0.5,
            prefix_padding_ms=300,
            silence_duration_ms=500)

        # Create session configuration with MCP tools in the tools list
        session_config = RequestSession(
            modalities=[Modality.TEXT, Modality.AUDIO],
            instructions=self.instructions,
```

```
            voice=voice_config,
            input_audio_format=InputAudioFormat.PCM16,
            output_audio_format=OutputAudioFormat.PCM16,
            turn_detection=turn_detection_config,
            input_audio_echo_cancellation=AudioEchoCancellation(),
            input_audio_noise_reduction=AudioNoiseReduc-
tion(type="azure_deep_noise_suppression"),
            tools=mcp_tools,
            tool_choice=ToolChoiceLiteral.AUTO,
            input_audio_transcription=AudioInputTranscriptionOptions(
                model="azure-speech" if "realtime" not in self.mod-
el.lower() else "whisper-1"
            ),
        )

        # Interim response bridges latency during MCP tool calls, but
is only
        # supported on non-realtime model pipelines (e.g. gpt-4o-mini).
        if "realtime" not in self.model.lower():
            session_config.interim_response = LlmInterimResponseConfig(
                triggers=[InterimResponseTrigger.TOOL, InterimResponse-
Trigger.LATENCY],
                latency_threshold_ms=100,
                instructions="Create friendly interim responses indi-
cating wait time due to "
                             "ongoing processing, if any. Do not in-
clude in all responses! "
                             "Do not say you don't have real-time ac-
cess to information when "
                             "calling tools!",
            )
            logger.info("Interim response enabled for model %s", self.-
model)
        else:
            logger.info("Interim response skipped — not supported on
realtime pipeline (%s)", self.model)

        conn = self.connection
        assert conn is not None
        await conn.session.update(session=session_config)
        logger.info("Session configuration with MCP tools sent")
    # </configure_session>

    async def _process_events(self):
        """Process events from the VoiceLive connection."""
        conn = self.connection
        assert conn is not None
        async for event in conn:
            try:
                await self._handle_event(event)
            except Exception:
                logger.exception("Error handling event %s (non-fatal)",
getattr(event, 'type', '?'))
```

```
    # <handle_mcp_events>
    async def _handle_event(self, event):
        """Handle different types of events from VoiceLive, including
MCP events."""
        ap = self.audio_processor
        conn = self.connection
        assert ap is not None
        assert conn is not None

        if event.type == ServerEventType.SESSION_UPDATED:
            logger.info("Session ready: %s", event.session.id)
            await write_conversation_log(f"SessionID: {event.ses-
sion.id}")
            await write_conversation_log(f"Model: {event.session.mod-
el}")
            await write_conversation_log(f"Voice: {event.ses-
sion.voice}")
            await write_conversation_log("")
            self.session_ready = True
            ap.start_capture()

        elif event.type == ServerEventType.INPUT_AUDIO_BUFFER_SPEECH_S-
TARTED:
            logger.info("User started speaking - stopping playback")
            print("🎤 Listening...")
            ap.skip_pending_audio()
            # Approval call counter is NOT reset on speech - it tracks
the
            # lifecycle of a task (reset on denial or after results are
spoken)
            # But approved-servers-this-turn resets when user starts a
new topic
            if self._pending_approval is None and self._mcp_cal-
l_in_progress <= 0:
                self._approved_servers_this_turn.clear()

            # Clear deferred response flags if no MCP calls are in
progress.
            # Prevents stale _needs_response_create from re-triggering
result
            # playback after the user interrupts.
            if self._mcp_call_in_progress <= 0:
                self._needs_response_create = False
                self._mcp_results_pending = False

            if self._active_response and not self._response_api_done:
                try:
                    await conn.response.cancel()
                except Exception as e:
                    if "no active response" not in str(e).lower():
                        logger.warning("Cancel failed: %s", e)

            # If an MCP call is running, mark current calls as stale
(user is moving on)
```

```
            # and let the user know it's still in progress
            if self._mcp_call_in_progress > 0 and self._pending_ap-
proval is None:
                self._stale_mcp_items.update(self._active_mcp_items)
                logger.info("User spoke during MCP call — marking %d
calls as stale", len(self._active_mcp_items))
                try:
                    await conn.conversation.item.create(
                        item=MessageItem(
                            role="system",
                            content=[InputTextContentPart(
                                text="A tool call is still running in
the background. The user just spoke. "
                                     "Respond to what the user said. If
a tool result arrives later, "
                                     "briefly introduce it as a late
result from an earlier request."
                            )],
                        )
                    )
                except Exception as e:
                    logger.warning("Failed to inject MCP status update:
%s", e)

        elif event.type == ServerEventType.INPUT_AUDIO_BUFFER-
_SPEECH_STOPPED:
            logger.info("User stopped speaking")
            print("🤔 Processing...")

        elif event.type == ServerEventType.RESPONSE_CREATED:
            logger.info("Assistant response created")
            self._active_response = True
            self._response_api_done = False

        elif event.type == ServerEventType.RESPONSE_AUDIO_DELTA:
            ap.queue_audio(event.delta)

        elif event.type == ServerEventType.RESPONSE_AUDIO_DONE:
            logger.info("Assistant finished speaking")
            print("🎤 Ready for next input...")

        elif event.type == ServerEventType.RESPONSE_TEXT_DONE:
            text = event.text if hasattr(event, 'text') else even-
t.get("text", "")
            print(f"🤖 Assistant text:\t{text}")
            await write_conversation_log(f"Assistant Text Re-
sponse:\t{text}")

        elif event.type == ServerEventType.RESPONSE_AUDIO_TRANSCRIPT_-
DONE:
            transcript = event.transcript if hasattr(event, 'tran-
script') else event.get("transcript", "")
            print(f"🤖 Assistant audio transcript:\t{transcript}")
            await write_conversation_log(f"Assistant Audio Re-
```

```
sponse:\t{transcript}")

        elif event.type == ServerEventType.RESPONSE_DONE:
            logger.info("Response complete")
            await write_conversation_log("--- Response complete ---")
            self._active_response = False
            self._response_api_done = True

            # If an approval prompt needs to be injected, do it now
that no response is active
            if self._approval_prompt_needed and self._pending_approval
is not None:
                self._approval_prompt_needed = False
                await self._send_approval_voice_prompt(self._pend-
ing_approval, conn)
            # If MCP results are pending and all calls are now done,
create response
            elif self._mcp_results_pending and self._mcp_cal-
l_in_progress <= 0 and self._pending_approval is None:
                self._mcp_results_pending = False
                try:
                    await conn.response.create()
                except Exception:
                    pass
            # If a response.create was deferred due to collision, retry
now
            elif self._needs_response_create:
                self._needs_response_create = False
                try:
                    await conn.response.create()
                except Exception:
                    pass  # Best-effort retry

        # <voice_approval_transcription>
        elif event.type == ServerEventType.CONVERSATION_ITEM_INPUT_AU-
DIO_TRANSCRIPTION_COMPLETED:
            transcript = event.transcript if hasattr(event, 'tran-
script') else event.get("transcript", "")
            logger.info("User said: %s", transcript)
            print(f"👤 You said:\t{transcript}")
            await write_conversation_log(f"User Input:\t{transcript}")

            # Interpret as an approval answer if we have a pending ap-
proval —
            # whether or not the prompt has finished speaking. This al-
lows the
            # user to barge in with "yes" without waiting for the full
prompt.
            if self._pending_approval is not None:
                await self._resolve_voice_approval(transcript, conn)
        # </voice_approval_transcription>

        elif event.type == ServerEventType.ERROR:
            msg = event.error.message
```

```
            # Reset response state — errors can terminate a response
without RESPONSE_DONE
            self._active_response = False
            self._response_api_done = True
            if "Cancellation failed: no active response" not in msg:
                if "interim response" in msg.lower():
                    logger.warning("Interim response not supported with
this model pipeline (non-fatal)")
                elif "active response" in msg.lower():
                    logger.debug("Response collision (expected during
MCP flow): %s", msg)
                else:
                    logger.error("VoiceLive error: %s", msg)
                    print(f"Error: {msg}")
                    await write_conversation_log(f"ERROR: {msg}")

        # MCP-specific events
        elif event.type == ServerEventType.MCP_LIST_TOOLS_IN_PROGRESS:
            logger.info("MCP list tools in progress for %s", even-
t.item_id)

        elif event.type == ServerEventType.MCP_LIST_TOOLS_COMPLETED:
            logger.info("MCP list tools completed for %s", even-
t.item_id)
            print("🔧 MCP tools discovered successfully")
            await write_conversation_log("MCP tools discovered success-
fully")

        elif event.type == ServerEventType.MCP_LIST_TOOLS_FAILED:
            logger.error("MCP list tools failed for %s", event.item_id)
            print("❌ MCP tool discovery failed")
            await write_conversation_log("ERROR: MCP tool discovery
failed")

        elif event.type == ServerEventType.RESPONSE_MCP_CAL-
L_IN_PROGRESS:
            logger.info("MCP call in progress for %s", event.item_id)
            print("⏳ MCP tool call in progress...")
            await write_conversation_log(f"MCP call in progress: {even-
t.item_id}")
            self._mcp_call_in_progress += 1
            self._active_mcp_items.add(event.item_id)
            self._start_mcp_stall_timer(conn)

        elif event.type == ServerEventType.RESPONSE_MCP_CALL_COMPLETED:
            item_id = event.item_id
            self._mcp_call_in_progress = max(0, self._mcp_cal-
l_in_progress - 1)
            self._active_mcp_items.discard(item_id)
            self._cancel_mcp_stall_timer()
            if item_id in self._handled_mcp_completions:
                logger.debug("Ignoring duplicate MCP completion for
%s", item_id)
            else:
```

```
                self._handled_mcp_completions.add(item_id)
                is_stale = item_id in self._stale_mcp_items
                self._stale_mcp_items.discard(item_id)
                logger.info("MCP call completed for %s (stale=%s)",
item_id, is_stale)
                await write_conversation_log(f"MCP call completed:
{item_id} (stale={is_stale})")
                await self._handle_mcp_call_completed(event, conn,
is_stale=is_stale)

        elif event.type == ServerEventType.RESPONSE_MCP_CALL_FAILED:
            item_id = event.item_id
            logger.error("MCP call failed for %s", item_id)
            print("❌ MCP tool call failed")
            await write_conversation_log(f"ERROR: MCP call failed:
{item_id}")
            self._mcp_call_in_progress = max(0, self._mcp_cal-
l_in_progress - 1)
            self._active_mcp_items.discard(item_id)
            self._stale_mcp_items.discard(item_id)
            self._cancel_mcp_stall_timer()
            # Kick the model to inform the user the tool call failed
            try:
                await conn.response.create()
            except Exception as e:
                if "active response" not in str(e).lower():
                    logger.warning("Failed to create response after MCP
failure: %s", e)

        elif event.type == ServerEventType.CONVERSATION_ITEM_CREATED:
            logger.info("Conversation item created: id=%s, type=%s",
event.item.id, event.item.type)
            if event.item.type == ItemType.MCP_LIST_TOOLS:
                logger.info("MCP list tools item: server_label=%s",
event.item.server_label)
            elif event.item.type == ItemType.MCP_CALL:
                await self._handle_mcp_call_arguments(event, conn)
            elif event.item.type == ItemType.MCP_APPROVAL_REQUEST:
                await self._handle_mcp_approval_request(event, conn)
        else:
            logger.debug("Unhandled event type: %s", event.type)
    # </handle_mcp_events>

    # <handle_approval>
    async def _handle_mcp_approval_request(self, conversation_creat-
ed_event, connection):
        """Handle MCP approval request by asking the user via voice."""
        if not isinstance(conversation_created_event, ServerEventCon-
versationItemCreated):
            logger.error("Expected ServerEventConversationItemCreated")
            return
        if not isinstance(conversation_created_event.item, ResponseMC-
PApprovalRequestItem):
            logger.error("Expected ResponseMCPApprovalRequestItem")
```

```
            return

        mcp_approval_item = conversation_created_event.item
        approval_id = mcp_approval_item.id
        server_label = mcp_approval_item.server_label
        function_name = mcp_approval_item.name

        if not approval_id:
            logger.error("MCP approval item missing ID")
            return

        # Auto-deny after too many calls to the same server in one
task.
        # This prevents infinite tool-call loops in voice UX.
        MAX_APPROVAL_CALLS_PER_TASK = 3
        current_count = self._approval_call_count.get(server_label, 0)
        if current_count >= MAX_APPROVAL_CALLS_PER_TASK:
            logger.info("Auto-denying %s - reached %d calls this task",
function_name, current_count)
            print(f"   Auto-denied: {server_label}/{function_name} (max
{MAX_APPROVAL_CALLS_PER_TASK} calls reached)")
            try:
                await connection.conversation.item.create(
                    item=MCPApprovalResponseRequestItem(approval_re-
quest_id=approval_id, approve=False)
                )
            except Exception as e:
                logger.warning("Failed to send auto-deny: %s", e)
            return

        # Auto-approve if user already approved this server earlier in
the same turn.
        # This avoids repeated approval prompts for consecutive calls
to the same service.
        if server_label in self._approved_servers_this_turn:
            logger.info("Auto-approving %s - server already approved
this turn", function_name)
            print(f"   Auto-approved: {server_label}/{function_name}
(already approved this turn)")
            try:
                await connection.conversation.item.create(
                    item=MCPApprovalResponseRequestItem(approval_re-
quest_id=approval_id, approve=True)
                )
            except Exception as e:
                logger.warning("Failed to send auto-approve: %s", e)
            return

        # If another approval is already pending, queue this one
        if self._pending_approval is not None:
            logger.info("Queuing approval for %s - another is already
pending", function_name)
            self._approval_queue.append({
                "approval_id": approval_id,
```

```
                "server_label": server_label,
                "function_name": function_name,
            })
            return

        logger.info("MCP approval request: server=%s tool=%s",
server_label, function_name)
        print(f"\n🔐 MCP Approval Request (voice-based):")
        print(f"   Server: {server_label}  Tool: {function_name}")

        # Store the pending approval. If no response is currently ac-
tive,
        # send the voice prompt immediately. Otherwise, defer it to
        # RESPONSE_DONE to avoid colliding with an active response.
        self._pending_approval = {
            "approval_id": approval_id,
            "server_label": server_label,
            "function_name": function_name,
        }

        if not self._active_response:
            await self._send_approval_voice_prompt(self._pending_ap-
proval, connection)
        else:
            self._approval_prompt_needed = True

    async def _send_approval_voice_prompt(self, pending: dict, connec-
tion):
        """Inject a system message asking the model to verbally request
permission."""
        server = pending["server_label"]
        call_count = self._approval_call_count.get(server, 0)
        self._approval_call_count[server] = call_count + 1

        if call_count == 0:
            prompt = (
                "You MUST ask the user for explicit permission before
proceeding. "
                f'Say exactly: "I\'d like to search the {server} ser-
vice for information. '
                f'Do you approve? Please say yes or no."'
            )
        else:
            prompt = (
                "You MUST ask the user for permission again. "
                'Say exactly: "I need to do one more search to get com-
plete information. '
                'Should I continue? Please say yes or no."'
            )

        try:
            await connection.conversation.item.create(
                item=MessageItem(
                    role="system",
```

```
                    content=[InputTextContentPart(text=prompt)],
                )
            )
            await connection.response.create()
        except Exception as e:
            logger.warning("Failed to send approval voice prompt: %s",
e)

    async def _resolve_voice_approval(self, transcript: str, connec-
tion):
        """Interpret the user's spoken response as approval or de-
nial."""
        pending = self._pending_approval
        if pending is None:
            return

        text = transcript.strip().lower()

        # Match "yes" or "no" as whole words (word boundaries prevent
false
        # positives from words like "yesterday" or "nobody").
        # Also accept "stop" and "cancel" as denial.
        approved = bool(re.search(r'\byes\b', text))
        denied = bool(re.search(r'\b(no|stop|cancel)\b', text))

        if not approved and not denied:
            # Ambiguous — ask again via the deferred prompt mechanism
            logger.info("Ambiguous approval response: %s", transcript)
            self._approval_prompt_needed = True
            return

        if approved and denied:
            # Conflicting signals — treat as denial for safety
            approved = False

        # Clear the pending state before sending the response
        self._pending_approval = None
        if approved:
            self._approved_servers_this_turn.add(pending["server_la-
bel"])
        else:
            self._approval_call_count.clear()  # Topic is over
            self._approved_servers_this_turn.discard(pend-
ing["server_label"])

        approval_response_item = MCPApprovalResponseRequestItem(
            approval_request_id=pending["approval_id"], approve=ap-
proved
        )
        try:
            await connection.conversation.item.create(item=approval_re-
sponse_item)
        except Exception as e:
            logger.error("Failed to send approval response: %s", e)
```

```
            return
        logger.info("Voice approval resolved: %s for %s", approved,
pending["function_name"])
        print(f"   Voice approval: {'Approved ✅' if approved else 'De-
nied ❌'}")
        await write_conversation_log(f"Voice approval: {'Approved' if
approved else 'Denied'} for {pending['server_label']}")

        # Process next queued approval, if any
        await self._process_next_approval(connection)

    async def _process_next_approval(self, connection):
        """Pop the next queued approval and ask via voice."""
        if not self._approval_queue:
            return
        next_approval = self._approval_queue.pop(0)
        self._pending_approval = next_approval

        # Send immediately if no response is active, otherwise defer
        if not self._active_response:
            await self._send_approval_voice_prompt(next_approval, con-
nection)
        else:
            self._approval_prompt_needed = True
    # </handle_approval>

    # <mcp_stall_detection>
    MCP_STALL_MAX_NOTIFICATIONS = 3

    def _start_mcp_stall_timer(self, connection):
        """Start a repeating timer that verbally updates the user if an
MCP call takes too long."""
        self._cancel_mcp_stall_timer()

        async def _stall_loop():
            stall_count = 0
            while self._mcp_call_in_progress > 0 and stall_count <
self.MCP_STALL_MAX_NOTIFICATIONS:
                await asyncio.sleep(10)
                if self._mcp_call_in_progress <= 0:
                    break
                stall_count += 1
                # Note: MCP calls cannot be cancelled via the API —
only honest
                # status updates are possible until the server responds
or times out.
                msg = ("The tool call is still running. "
                       "Briefly reassure the user that you're still
waiting for results. "
                       "One short sentence only.")
                logger.info("MCP stall notification #%d", stall_count)
                try:
                    await connection.conversation.item.create(
                        item=MessageItem(
```

```
                            role="system",
                            content=[InputTextContentPart(text=msg)],
                        )
                    )
                    await connection.response.create()
                except Exception as e:
                    if "active response" in str(e).lower():
                        self._needs_response_create = True
                    else:
                        logger.debug("Stall notification failed: %s",
e)

        self._mcp_stall_task = asyncio.create_task(_stall_loop())

    def _cancel_mcp_stall_timer(self):
        """Cancel the MCP stall timer if running."""
        if self._mcp_stall_task and not self._mcp_stall_task.done():
            self._mcp_stall_task.cancel()
        self._mcp_stall_task = None
    # </mcp_stall_detection>

    async def _handle_mcp_call_completed(self, mcp_call_complet-
ed_event, connection, *, is_stale=False):
        """Handle MCP call completed events."""
        if not isinstance(mcp_call_completed_event, ServerEventRespon-
seMcpCallCompleted):
            logger.error("Expected ServerEventResponseMcpCallComplet-
ed")
            return

        logger.info("MCP call completed for %s (stale=%s)", mcp_call_-
completed_event.item_id, is_stale)
        print("✅ MCP tool call completed successfully")

        # Clean up item mapping
        self._mcp_item_to_server.pop(mcp_call_completed_event.item_id,
None)

        # Reset approval counter if no more approvals are pending (task
complete)
        if self._pending_approval is None and not self._approval_queue:
            self._approval_call_count.clear()

        # If the user moved on during this call, tell the model it's a
late result
        if is_stale:
            try:
                await connection.conversation.item.create(
                    item=MessageItem(
                        role="system",
                        content=[InputTextContentPart(
                            text="This tool result is from an earlier
request. The user has "
                                 "since moved on. Briefly introduce it
```

```
as a late result, e.g. "
                                "'By the way, those results from ear-
lier just came in...' "
                                "then share the key findings concise-
ly."
                            )],
                        )
                    )
                except Exception as e:
                    logger.warning("Failed to inject late-result context:
%s", e)

            # Batch response: only call response.create when ALL MCP calls
for this
            # turn have completed. This prevents partial results and re-
peated tool calls.
            if self._mcp_call_in_progress <= 0 and self._pending_approval
is None and not self._approval_queue:
                logger.info("All MCP calls complete - creating response")
                try:
                    await connection.response.create()
                except Exception as e:
                    if "active response" in str(e).lower():
                        self._needs_response_create = True
                    else:
                        logger.warning("Failed to create response after MCP
calls: %s", e)
            else:
                self._mcp_results_pending = True
                logger.info("MCP calls still in progress (%d) - deferring
response", self._mcp_call_in_progress)

    async def _handle_mcp_call_arguments(self, conversation_creat-
ed_event, connection):
            """Log MCP call details and announce the tool call to the user
via voice."""
            if not isinstance(conversation_created_event, ServerEventCon-
versationItemCreated):
                logger.error("Expected ServerEventConversationItemCreated")
                return
            if not isinstance(conversation_created_event.item, ResponseMCP-
CallItem):
                logger.error("Expected ResponseMCPCallItem")
                return

            mcp_call_item = conversation_created_event.item
            server_label = mcp_call_item.server_label
            function_name = mcp_call_item.name

            logger.info("MCP Call triggered: server_label=%s, function_-
name=%s", server_label, function_name)
            print(f"🔧 MCP tool call: {server_label}/{function_name}")
            self._mcp_item_to_server[mcp_call_item.id] = f"{server_la-
bel}/{function_name}"
```

```
        # Announce the tool call to the user so they know something is
        # happening while the MCP call runs. Skip for approval-required
        # servers (the approval prompt handles communication) and skip
        # if an approval is already pending.
        if self._pending_approval is None and server_label not in
self._approval_servers:
            try:
                await connection.conversation.item.create(
                    item=MessageItem(
                        role="system",
                        content=[InputTextContentPart(
                            text="Briefly tell the user you're looking
something up. One short sentence only."
                        )],
                    )
                )
                await connection.response.create()
            except Exception as e:
                if "active response" not in str(e).lower():
                    logger.warning("Failed to create tool announcement:
%s", e)


def parse_arguments():
    """Parse command line arguments."""
    parser = argparse.ArgumentParser(
        description="Voice Assistant with MCP using Azure VoiceLive
SDK",
    )

    parser.add_argument(
        "--api-key",
        help="Azure VoiceLive API key (or set AZURE_VOICELIVE_API_KEY
env var)",
        type=str,
        default=os.environ.get("AZURE_VOICELIVE_API_KEY"),
    )
    parser.add_argument(
        "--endpoint",
        help="Azure VoiceLive endpoint (default: from AZURE_VOICELIV-
E_ENDPOINT env var)",
        type=str,
        default=os.environ.get("AZURE_VOICELIVE_ENDPOINT",
"https://your-resource-name.services.ai.azure.com/"),
    )
    parser.add_argument(
        "--model",
        help="VoiceLive model to use (default: gpt-realtime)",
        type=str,
        default=os.environ.get("AZURE_VOICELIVE_MODEL", "gpt-real-
time"),
    )
    parser.add_argument(
```

```
        "--voice",
        help="Voice to use for the assistant (default: en-US-Ava:DragonHDLatestNeural)",
        type=str,
        default=os.environ.get("AZURE_VOICELIVE_VOICE", "en-US-Ava:DragonHDLatestNeural"),
    )
    parser.add_argument(
        "--instructions",
        help="System instructions for the AI assistant",
        type=str,
        default=os.environ.get(
            "AZURE_VOICELIVE_INSTRUCTIONS",
            "You are a helpful AI assistant with access to MCP tools. "
            "Always respond in English. "
            "When a user asks a question, use the appropriate tool once to find information, "
            "then summarize the results conversationally. IMPORTANT: Never call the same tool "
            "more than once per user question. After receiving a tool result, always respond "
            "to the user with what you found — do not search again. "
            "Some tools require user approval before they can be used. When you receive a "
            "system message asking you to request permission, you MUST clearly ask the user "
            "for their explicit approval before proceeding. Always wait for the user to say "
            "yes or no. Never skip the approval question or assume permission is granted. "
            "If a tool result arrives after the conversation has moved to a different topic, "
            "briefly introduce it as a late result before sharing the findings.",
        ),
    )
    parser.add_argument(
        "--use-token-credential", help="Use Azure token credential instead of API key", action="store_true", default=False
    )
    parser.add_argument("--verbose", help="Enable verbose logging", action="store_true")

    return parser.parse_args()


async def write_conversation_log(message: str) -> None:
    """Write a message to the conversation log."""
    log_path = os.path.join(_script_dir, 'logs', conversation_logfilename)
    def _write():
        with open(log_path, 'a', encoding='utf-8') as f:
            f.write(message + "\n")
    await asyncio.to_thread(_write)
```

```python
def main():
    """Main function."""
    args = parse_arguments()

    if args.verbose:
        logging.getLogger().setLevel(logging.DEBUG)

    if not args.api_key and not args.use_token_credential:
        print("❌ Error: No authentication provided")
        print("Please provide an API key using --api-key or set AZURE_VOICELIVE_API_KEY environment variable,")
        print("or use --use-token-credential for Azure authentication.")
        sys.exit(1)

    credential: Union[AzureKeyCredential, AsyncTokenCredential]
    if args.use_token_credential:
        credential = AzureCliCredential()
        logger.info("Using Azure token credential")
    else:
        credential = AzureKeyCredential(args.api_key)
        logger.info("Using API key credential")

    assistant = MCPVoiceAssistant(
        endpoint=args.endpoint,
        credential=credential,
        model=args.model,
        voice=args.voice,
        instructions=args.instructions,
    )

    def signal_handler(_sig, _frame):
        logger.info("Received shutdown signal")
        raise KeyboardInterrupt()

    signal.signal(signal.SIGINT, signal_handler)
    signal.signal(signal.SIGTERM, signal_handler)

    try:
        asyncio.run(assistant.start())
    except KeyboardInterrupt:
        print("\n👋 Voice assistant with MCP shut down. Goodbye!")
    except Exception as e:
        print("Fatal Error: ", e)


if __name__ == "__main__":
    # Check audio system
    try:
        p = pyaudio.PyAudio()
        input_devices = [
            i for i in range(p.get_device_count())
```

```
            if cast(Union[int, float], p.get_device_info_by_in-
dex(i).get("maxInputChannels", 0) or 0) > 0
        ]
        output_devices = [
            i for i in range(p.get_device_count())
            if cast(Union[int, float], p.get_device_info_by_in-
dex(i).get("maxOutputChannels", 0) or 0) > 0
        ]
        p.terminate()

        if not input_devices:
            print("❌ No audio input devices found. Please check your
microphone.")
            sys.exit(1)
        if not output_devices:
            print("❌ No audio output devices found. Please check your
speakers.")
            sys.exit(1)
    except Exception as e:
        print(f"❌ Audio system check failed: {e}")
        sys.exit(1)

    print("🎙️  Voice Assistant with MCP - Azure VoiceLive SDK")
    print("=" * 65)

    main()
```

2. Sign in to Azure with the following command:

```shell
az login
```

3. Run the Python script:

```shell
python mcp-quickstart.py
```

4. Speak into your microphone. Try asking questions like "What tools do you have?" or "Search the Azure documentation for Voice Live API."

    - For the `deepwiki` server (`require_approval="never"`), tool calls execute automatically.
    - For the `azure_doc` server (`require_approval="always"`), you're prompted to approve each tool call in the console.

5. Press **Ctrl+C** to stop the session.

MCP server configuration reference

Expand table

Parameter	Required	Description
`server_label`	Yes	Display name for the MCP server.
`server_url`	Yes	URL of the remote MCP endpoint.
`allowed_tools`	No	List of tool names the model can call. If omitted, all tools are allowed.
`require_approval`	No	`"never"`, `"always"` (default), or a per-tool dictionary.
`headers`	No	Extra HTTP headers to include in MCP requests.
`authorization`	No	Authorization token for MCP requests.

For the complete REST API type definition, see MCPTool in the Voice Live API reference.

Best practices

Integrating MCP servers into a voice assistant introduces UX challenges that don't exist in text-based or console-based MCP clients. MCP tool calls can take 3–60+ seconds, approval prompts must happen conversationally, and users expect continuous spoken feedback. Plan for these patterns when building a voice-enabled MCP integration.

Voice-native approval

Console-based MCP samples typically use blocking input (such as `input()` or `readline`) for approval. In a voice assistant, blocking the audio pipeline freezes the conversation. Instead, handle approvals conversationally:

- Inject a system message that instructs the model to **verbally ask for permission**.
- Parse the user's spoken response for clear intent (`yes`, `no`, `stop`, `cancel`).
- Allow **barge-in** so the user can say "yes" without waiting for the full approval prompt to finish.
- Use word-boundary matching (such as `\byes\b`) to avoid false positives from words like

"yesterday" or "nobody".

System instructions for the approval flow

The model needs explicit instructions about the approval flow in its system prompt. Without them, it might paraphrase the permission request into a generic "Let me look that up," skipping the actual question. Include language like:

> *"Some tools require user approval. When you receive a system message asking you to request permission, you MUST clearly ask the user for their explicit approval. Never skip the approval question or assume permission is granted."*

Use `"Say exactly:"` phrasing in per-request system messages to prevent the model from rewording the question.

Handle repeated tool calls

MCP servers might require multiple searches to gather complete information. Each search triggers a separate approval if `require_approval="always"`. Rather than asking the identical question each time:

- Track the call count per server.
- Change the prompt wording for subsequent calls (for example, "I need one more search. Should I continue?").
- Consider auto-denying after a maximum number of approved calls (for example, 3) to prevent infinite loops. The model responds with what it has.
- Reset the counter when results are delivered or the user denies a request.

For approval-required servers, consider auto-approving subsequent calls to the **same server within the same turn** to avoid repeated voice prompts for what is logically a single task.

Fill silence during tool calls

MCP tool calls can take several seconds to complete. Without feedback, the user assumes the assistant is unresponsive. Use these complementary layers:

1. **Tool announcements** (immediate, client-side): For auto-approved servers, have the

assistant say something like "Let me look that up" when the call starts. Skip this for approval-required servers since the approval prompt already communicates that a tool call is happening.

2. **Stall detection** (client-side, repeating timer): If a tool call runs longer than expected, proactively tell the user the assistant is still waiting. A 10-second interval with a maximum of 3 notifications works well for medium-latency servers (5–15 seconds). Adjust the interval based on your expected MCP server latency.

> ⓘ **Note**
>
> MCP calls can't be cancelled. Stall notifications are status updates, not actionable options. Once a call starts, it runs until the server responds or times out.

Handle barge-in during MCP calls

Users naturally try to interrupt or ask "Are you still there?" during long tool calls. Rather than ignoring this:

- Inject a system message so the model can acknowledge the user.
- If the original MCP call completes later, introduce its result as a late result (for example, "By the way, those results from earlier just came in...").
- Protect against response collisions: when a cancelled response's completion handler runs, skip any deferred processing (pending approval prompts, queued MCP results) so it doesn't overlap with the user's new turn.

Choose MCP servers for voice latency

Not all MCP servers are well-suited for voice UX. When selecting MCP servers for a voice assistant:

- **Prefer low-latency servers** — search APIs, simple lookups, and cached data sources that respond within 5 seconds work best.
- **Avoid servers that perform heavy computation** — large repository analysis, complex document retrieval, or multi-step workflows can take 30–60+ seconds, degrading the voice experience.
- **Plan for non-cancellable calls** — MCP calls can't be cancelled from the client. If the user

moves on during a slow call, the result arrives out of context and must be introduced as a late result, which can feel disjointed.

- **Consider your use case** — if users expect real-time answers, long-running MCP servers frustrate them. If the interaction style is more like a research assistant, asynchronous results might be acceptable.

Troubleshooting

MCP tool discovery fails (`mcp_list_tools.failed`)

Voice Live contacts each MCP server's tool listing endpoint at session start. If discovery fails, no tools from that server are available during the session.

Expand table

Cause	Resolution
Incorrect `server_url`	Verify the MCP server URL is reachable and includes the correct path (for example, `https://mcp.deepwiki.com/mcp`).
Server is unreachable	Confirm the MCP server is running and accessible from Azure's network. Check firewall rules and DNS resolution.
Authentication failure	If the server requires authentication, verify the `authorization` or `headers` values are correct and not expired.
Server returns invalid tool schema	Check the MCP server's tool listing response conforms to the MCP specification.

MCP tool call fails (`response.mcp_call.failed`)

A tool call failure means Voice Live successfully discovered the tool but the call didn't complete.

Expand table

Cause	Resolution
Server timeout	The MCP server took too long to respond. Optimize the server-side handler or choose a lower-latency server.

Server returned an error	Check your MCP server logs. Common issues include missing parameters, invalid input, or downstream service failures.
Network interruption	Transient network errors between Voice Live and the MCP server. Retry by prompting the model again.

> **Tip**
>
> When an MCP call fails, trigger `response.create` so the model can inform the user and continue the conversation. The sample code does this automatically.

No MCP events received

Expand table

Cause	Resolution
Wrong API version	MCP requires `api_version="2026-01-01-preview"` or later. Earlier API versions silently ignore MCP server configuration.
MCP servers not in session config	Verify that `MCPServer` objects are included in the `tools` list passed to `configure_session` or `updateSession`.
`allowed_tools` mismatch	If `allowed_tools` is set, only the listed tool names are exposed. Verify the names match exactly what the MCP server advertises.

Approval requests not received

Expand table

Cause	Resolution
`require_approval` set to `"never"`	Tool calls auto-execute without approval. Change to `"always"` or use a per-tool dictionary if you need approval for specific tools.
Event handler not subscribed	Ensure your code listens for `mcp_approval_request` conversation items in the event loop.
Duplicate handling	The approval request arrives as a conversation item creation event, not a standalone event type. Check that your `conversation.item.created` handler inspects the item type.

Response collision errors during MCP flow

Voice Live doesn't allow overlapping responses. During MCP flows, `response.create` calls can collide with an in-progress response.

Expand table

Cause	Resolution
`"Cancellation failed: no active response"`	Non-fatal. This occurs when a cancel is issued but the response already completed. Log and ignore.
`"active response"` errors	A new `response.create` was attempted while another response is still generating. Track response state (`response.created` / `response.done` events) and defer actions until the active response completes.
Interim response errors	Some model pipelines don't support `interimResponse`. If you receive interim response errors, remove the interim response configuration or verify your model supports it.

Related content

- Function calling in Voice Live
- How to build a voice agent
- Voice Live API reference (2026-01-01-preview)
- Voice Live quickstart

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 04/14/2026